# MarketSight

Stock Market Prediction System

Final Configuration Management Plan

**Prepared by:** Zane Tessmer (Scrum Master)

COSC 471 - Winter 2026 | Okanagan College

March 17, 2026

# 1. Introduction

## 1.1 Purpose

This Configuration Management (CM) Plan defines the policies, processes, and tools used to manage all configuration items in the MarketSight project. It ensures that every artifact - source code, model weights, documentation, and infrastructure configuration - is version-controlled, traceable, and reproducible.

## 1.2 Scope

This plan covers all software artifacts, model artifacts, infrastructure configurations, database schemas, and documentation for the MarketSight stock market prediction system.

## 1.3 Tools

| Tool | Purpose |
|---|---|
| **GitHub** | Version control, issue tracking, pull requests, CI/CD |
| **Docker / Docker Compose** | Containerization, reproducible environments |
| **Alembic** | Database schema migration management |
| **GitHub Actions** | Continuous integration and automated testing |
| **Airflow** | Pipeline scheduling and workflow management |

# 2. Version Management

## 2.1 Version Control System

- **Platform:** GitHub
- **Repository:** `Okanagan-College-Cosc471-Winter-2026/the-project-maverick`
- **Access:** All team members have push access; `main` branch is protected

## 2.2 Branching Strategy

The project uses a **feature branch workflow**:

```
main (protected)
  |
  +-- ft-airflow         (merged Sprint 4)
  +-- ft-tests           (merged Sprint 4)
  +-- ft-Ml-training     (merged Sprint 5, PR #48)
  +-- ft-dri-migration   (merged Sprint 5, PR #49)
  +-- dante_feature      (open PR #45)
```

### Branch Naming Convention

- `ft-<feature-name>` - Feature branches (e.g., `ft-airflow`, `ft-Ml-training`)
- `dante_feature` - Developer-specific feature branches
- `copilot/<description>` - Auto-generated documentation branches

### Branch Lifecycle

1. Create branch from `main`
2. Develop with atomic commits using conventional commit messages
3. Open Pull Request targeting `main`
4. CI runs automatically (6 jobs)
5. At least one peer review required
6. Merge coordination meeting for shared module changes
7. Merge to `main` after approval and passing CI
8. Delete feature branch after merge

## 2.3 Commit Message Convention

All commits follow the **Conventional Commits** format:

```
<type>: <description>

Types:
```

```
feat:     New feature
fix:      Bug fix
refactor: Code restructuring
docs:     Documentation changes
test:     Test additions/changes
chore:    Build, CI, dependency changes
```

## 2.4 Version Numbering

| Component | Current Version | Convention |
|---|---|---|
| Application | v1.1.0 | Semantic versioning (major.minor.patch) |
| ML Model | xgboost-v1-{split_date} | Algorithm-version-training date |
| API | v1 | URL-based ( `/api/v1/...` ) |
| Database Schema | Alembic revision hash | Sequential migration revisions |

# 3. System Building

## 3.1 Build Process

All services are containerized with Docker. The build is defined in `docker-compose.yml`:

| Service | Image | Build Context | Port |
|---|---|---|---|
| backend | marketsight-backend | `./backend` | 8000 |
| frontend | marketsight-frontend | `./frontend` | 5173 |
| postgres | postgres:16 | Official image | 5432 |
| nginx | nginx:latest | `./docker/nginx` | 80, 443 |
| airflow-webserver | apache/airflow | `./airflow` | 8080 |
| airflow-scheduler | apache/airflow | `./airflow` | - |
| streamlit | marketsight-streamlit | `./frontend_streamlit` | 8501 |

### Build Commands

```
# Full stack (development)
docker compose up --build

# Individual service rebuild
docker compose up --build backend

# Production deployment (DRI)
docker compose -f docker-compose.yml up -d
```

## 3.2 License Information

| Dependency | License | Category |
|---|---|---|
| FastAPI | MIT | Backend framework |
| React 19 | MIT | Frontend framework |

| | | |
|---|---|---|
| XGBoost | Apache 2.0 | ML library |
| Optuna | MIT | HPO framework |
| PostgreSQL 16 | PostgreSQL License | Database |
| SQLAlchemy 2.0 | MIT | ORM |
| TanStack Router | MIT | Frontend routing |
| Tailwind CSS | MIT | CSS framework |
| Lightweight Charts | Apache 2.0 | Charting library |
| Apache Airflow | Apache 2.0 | Pipeline scheduler |
| Streamlit | Apache 2.0 | Data app framework |
| Nginx | BSD-2-Clause | Reverse proxy |
| Docker | Apache 2.0 | Containerization |

## 3.3 Coding Standards

### Python (Backend)

- **Formatter:** Ruff (line length: 88)
- **Linter:** Ruff (isort, flake8, pyflakes rules)
- **Type Checker:** Mypy (strict mode)
- **Style:** PEP 8 compliant

### TypeScript (Frontend)

- **Formatter:** Biome
- **Linter:** Biome (recommended rules)
- **Style:** Functional components, hooks-based state management

### SQL

- **Migrations:** Alembic with auto-generated revisions
- **Raw SQL:** Used only in performance-critical OHLC queries
- **ORM:** SQLAlchemy 2.0 declarative models for all other queries

# 4. Release Management

## 4.1 Release History

| Release | Date | Sprint | Key Features |
|---|---|---|---|
| v0.1.0 | Feb 9, 2026 | Sprint 3 | Project scaffold, Docker env, ML notebook, initial frontend |
| v1.0.0 | Mar 3, 2026 | Sprint 4 | Inference API, confidence scoring, FMP pipeline, Airflow, charts |
| v1.1.0 | Mar 17, 2026 | Sprint 5 | DRI deployment, Streamlit, native model format, restored features |

## 4.2 Release Process

1. All feature branches merged to `main` via reviewed PRs
2. CI pipeline passes all 6 jobs on `main`
3. Acceptance tests executed and documented
4. Git tag created for the release (e.g., `v1.1.0`)
5. Docker images tagged with release version
6. Production deployment updated on DRI server
7. Release notes prepared and included in documentation

## 4.3 Deployment Environments

| Environment | URL | Method | Managed By |
|---|---|---|---|
| Local Dev | `localhost:5173` / `:8000` | `docker compose up` | Each developer |
| CI | GitHub Actions | Automated on push/PR | Automated |
| DRI Production | Production URL | Docker Compose + management scripts | Harsh Kumar |
| DRAC HPC | Fir cluster | Job scheduler (Slurm) | Harsh Kumar |

## 4.4 Model Artifact Management

ML model artifacts follow a structured format:

```
ml/models/
  meta.json              # Model metadata (horizon, split date, features, tickers)
  stock_prediction/
    model.ubj            # Native XGBoost binary format
    ticker_encoder.pkl   # Fitted label encoder for ticker symbols
```

Artifacts are versioned via:

- `meta.json` contains training date and split date
- Model version string format: `xgboost-v1-{split_date}`
- Backups created by Airflow with timestamps before retraining

# 5. Change Management

## 5.1 Change Request Process

1. Change requester creates a GitHub Issue or raises the change in standup
2. Product Owner (Harsh) assesses priority and impact
3. Team discusses in sprint planning or ad-hoc meeting
4. If approved, change is added to the sprint backlog with user story and points
5. Change is documented in the Change Request document with CR-ID
6. Implementation follows standard branch -> PR -> review -> merge flow
7. Change Request document is updated with implementation status

## 5.2 Change Request Format

| Field | Description |
|---|---|
| CR ID | Unique identifier (e.g., CR-001) |
| Title | Brief description of the change |
| Requested By | Person or role requesting the change |
| Date Submitted | When the request was made |
| Priority | High / Medium / Low |
| Status | Submitted / Approved / Implemented / Rejected |
| Description | Detailed description of the change |
| Justification | Why the change is needed |
| Impact | What systems/code are affected |
| Approval | Who approved and when |

## 5.3 Change Requests (Sprint 4-5)

| CR ID | Title | Status |
|---|---|---|
| CR-001 | Remove authentication module from MVP | Approved & Implemented |

| | | |
|---|---|---|
| CR-002 | Add prediction confidence scores to API | Approved & Implemented |
| CR-003 | Restore confidence scoring deleted during ML pipeline merge | Approved & Implemented |
| CR-004 | Migrate from JSON model format to native XGBoost .ubj | Approved & Implemented |
| CR-005 | Add Streamlit as secondary frontend | Approved & Implemented |

## 5.4 Configuration Item Inventory

| Item | Location | Owner | Version | Change Frequency |
|---|---|---|---|---|
| Backend source | `backend/app/` | Harsh Kumar | HEAD | High |
| Frontend (React) | `frontend/src/` | Parag Jindal | HEAD | High |
| Frontend (Streamlit) | `frontend_streamlit/` | Harsh Kumar | HEAD | Medium |
| ML scripts | `ml/scripts/` | Harsh Kumar | HEAD | Medium |
| Model artifacts | `ml/models/` | Harsh Kumar | meta.json | Low |
| Docker config | `docker-compose.yml` | Harsh Kumar | HEAD | Low |
| CI/CD | `.github/workflows/` | Harsh Kumar | HEAD | Low |
| DB migrations | `backend/alembic/versions/` | Harsh Kumar | Revision | Medium |
| Server docs | `docs_server_setup.md` | Harsh Kumar | HEAD | Low |
| DW sync | `scripts/` | Harsh Kumar | HEAD | Low |
| Env config | `.env` (not committed) | All | N/A | Low |
| Documentation | `Documentation/` | Zane Tessmer | HEAD | Medium |